

or those resources that you wish to create afresh using the commands `terraform destroy` and `terraform taint`, respectively. The easy way to get quick information about the Terraform commands and their options/arguments is by typing `terraform` and `terraform <command name> -h`.

The recent versions of Terraform have started to provide data sources, which are the resources to gather dynamic information from the various providers. The dynamic information gathered through the data sources is used in the Terraform configurations, most commonly using interpolation. A simple example of a data source is to gather the `ami id` for the latest version of an `ami` and use that in the instance provisioning configurations as shown below:

```
data "aws_ami" "myami" {
  most_recent = true

  filter {
    name = "name"
    values = ["MyBaseImage"]
  }
}

resource "aws_instance" "myvm" {
  ami = "${data.aws_ami.myami.id}"
}
```

Code organisation and reusability

Although our examples show the entire declarative configuration in a single file, we should break it into more than one file. You could break your whole config into various separate configs based upon the respective functionality they provide. So our first example could be broken into `variables.tf` that keeps all the variables blocks, `aws.tf` that declares our provider, `instances.tf` that declares the layout of the AWS VMs, `route53.tf` that declares the `aws route 53` functionality, and `output.tf` for our outputs. To keep things simple, use and maintain, keep everything related to a whole task being solved by Terraform in a single directory along with sub-directories that are named as files, scripts, keys, etc. Terraform doesn't enforce any hierarchy of code organisation, but keeping each high level functionality in its dedicated directory will save you from unexpected Terraform actions in spite of unrelated configuration changes. Remember, in the software world, "A little copying is better than a little dependency," as things get fragile and complicated easily with each added functionality.

Terraform provides the functionality of creating modules to reuse the configs created. The cluster creation template shown above is actually put in a

module to use the same code to provision test and/or production clusters. The usage of the module is simply supplying the required variables to it in the manner shown below (after running `terraform get` to create the necessary link for the module code):

```
module "myvms" {
  source = "../modules/awsvms"
  ami_id = "${var.ami_id}"
  inst_type = "${var.inst_type}"
  key_name = "${var.key_name}"
  subnet_id = "${var.subnet_id}"
  sg_id = "${var.sg_id}"
  num_nds = "${var.num_nds}"
  hst_env = "${var.hst_env}"
  apps_pckd = "${var.apps_pckd}"
  hst_rle = "${var.hst_rle}"
  root_size = "${var.root_size}"
  swap_size = "${var.swap_size}"
  vol_size = "${var.vol_size}"
  zone_id = "${var.zone_id}"
  prov_script = "${var.prov_script}"
  sub_dmn = "${var.sub_dmn}"
}
```

You also need to create a `variables.tf` in the location of your module source, requiring the same variables you fill in your module. Here is the module `variables.tf` to pass the variables supplied from the caller of the module:

```
variable "ami_id" {}
variable "inst_type" {}
variable "key_name" {}
variable "subnet_id" {}
variable "sg_id" {}
variable "num_nds" {}
variable "hst_env" {}
variable "apps_pckd" {}
variable "hst_rle" {}
variable "root_size" {}
variable "swap_size" {}
variable "vol_size" {}
variable "zone_id" {}
variable "prov_script" {}
variable "sub_dmn" {}
```

The Terraform official documentation consists of a few detailed sections for modules usage and creation, which should provide you more information on everything related to modules.

Importing existing resources

As we have seen earlier, Terraform caches the properties of the resources it creates into a `state` file, and by default doesn't know about the resources not created through it.